TEMPLE COLLEGE
WORKFORCE DEVELOPMENT

CHIEFOS

STUDENT REVIEW PACKET

Week 4

Networking, processes, pipes & grep, users — Uplink Hunt + Module 01 checkpoint

Phase 1 — Foundations · Weeks 1–6 · Modules 00–02

No D2L login required. This is the same lesson text posted in Brightspace for this week. Work the lessons in order on your own machine. Keep each checkpoint screenshot or pasted output. Bring it to the live session, or email it to your instructor, instead of submitting in D2L. Module quizzes stay in D2L and are not included here.

In this packet

1. 01.07 Networking Drills
2. 01.08 Processes and Services
3. 01.09 Pipes and Grep
4. 01.10 User and Group Setup
5. Live lab — Operation Uplink
6. 01.11 Final Checkpoint

Learning objective: Diagnose network readiness for Docker Hub using `ip a`, `ping`, `curl`, `ss -tuln`, and `dig`.

Lesson 07: Networking Drills

By the end, you will have confirmed your network is ready to pull Docker images.

~10 minutes of typing

---

1. Start from home.

```
cd ~
```

2. Confirm.

```
pwd
```

```
/home/your-username
```

3. View your network interfaces and IP addresses.

```
ip a
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> ...
    inet 127.0.0.1/8 scope host lo
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> ...
    inet 192.168.x.x/24 ...
```

(Output will vary. Look for an `inet` address on your main interface — that's your local IP.)

4. Ping Google's DNS to confirm basic internet connectivity.

```
ping -c 3 8.8.8.8
```

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=12.3 ms
...
3 packets transmitted, 3 received, 0% packet loss
```

5. Ping a domain name to confirm DNS works.

```
ping -c 3 google.com
```

```
PING google.com (142.250.x.x) 56(84) bytes of data.
...
3 packets transmitted, 3 received, 0% packet loss
```

6. Ping Docker Hub's domain — this is where we'll pull images from.

```
ping -c 3 hub.docker.com
```

(You should get 0% packet loss. If this fails, your network may block Docker Hub.)

7. Use curl to check HTTP headers from a website.

```
curl -I https://www.google.com
```

```
HTTP/2 200
content-type: text/html; charset=ISO-8859-1
...
```

(Look for `HTTP/2 200` — that means the request succeeded.)

8. Find your public IP address.

```
curl ifconfig.me
```

```
203.0.113.x
```

(This shows your public-facing IP. It will differ from your local IP.)

9. Add a newline after the IP (curl doesn't always add one).

```
echo ""
```

10. Check what ports are currently listening on your machine.

```
ss -tuln
```

```
Netid  State   Recv-Q  Send-Q   Local Address:Port   Peer Address:Port
tcp    LISTEN  0       128      0.0.0.0:22            0.0.0.0:*
```

(You should see at least port 22 for SSH. After Docker is installed, you'll see more ports here.)

11. Install dnsutils if dig isn't available yet.

```
sudo apt install -y dnsutils
```

12. Look up DNS records for Docker Hub.

```
dig hub.docker.com
```

```
;; ANSWER SECTION:
hub.docker.com.    300    IN    A    x.x.x.x
...
;; Query time: X msec
```

(This confirms your system can resolve DNS names — critical for pulling Docker images.)

13. Look up just the short answer.

```
dig +short hub.docker.com
```

```
x.x.x.x
```

14. Test a curl download — this is how Docker's install script works.

```
curl -fsSL https://www.google.com > /dev/null && echo "Download OK"
```

```
Download OK
```

(The -fsSL flags: f=fail silently, s=silent, S=show errors, L=follow redirects.)

15. Log your network status in your notes.

```
echo "Network check passed: $(date)" >> ~/docker-course/notes/notes.txt
```

16. Verify.

```
tail -n 1 ~/docker-course/notes/notes.txt
```

```
Network check passed: Mon Jan  1 12:00:00 UTC 2024
```

(Date will differ.)

Checkpoint

```
ping -c 1 8.8.8.8
```

You should see `1 packets transmitted, 1 received, 0% packet loss`.

```
curl -I https://hub.docker.com 2>/dev/null | head -n 1
```

You should see an HTTP status code (200 or 301).

```
dig +short google.com
```

You should see one or more IP addresses.

```
ss -tuln | head -n 5
```

You should see a table of listening ports.

Lab

Objective: Confirm your network can reach the internet, resolve DNS names, and speak HTTP — including to Docker Hub.

Task: Work through the networking drills (ping, curl, dig, ss), then run the four Checkpoint commands in one terminal.

Submit: One screenshot (or pasted transcript) of the four Checkpoint commands and their output: `ping -c 1 8.8.8.8`, `curl -I https://hub.docker.com 2>/dev/null | head -n 1`, `dig +short google.com`, `ss -tuln | head -n 5`.

Pass criteria: (network output is non-deterministic — verify the final state, never exact text. On campus/corporate networks `curl ifconfig.me` may show a proxy IP and some hosts may be filtered; any successful response counts.)

- [ ] `ping -c 1 8.8.8.8` shows `0% packet loss`.
- [ ] The `curl -I` check prints an HTTP status line (`HTTP/2 200`, a `301`, or similar — any status line back from the server is a successful response).
- [ ] `dig +short google.com` returns at least one IP address.
- [ ] `ss -tuln | head -n 5` shows the listening-ports table.

Unlocks: 08-processes-and-services.md

Next up: 08-processes-and-services.md — Inspect what's running

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Inspect running processes and service status using `ps aux`, `top`, `htop`, `systemctl`, `journalctl`, `uptime`, `free -h`, and `df -h`.

Lesson 08: Processes and Services

By the end, you will be able to inspect running processes and check service status — skills you'll need for managing Docker containers.

~15 minutes of typing

---

1. Start from home.

```
cd ~
```

2. Confirm.

```
pwd
```

```
/home/your-username
```

3. List all running processes.

```
ps aux
```

(You'll see a long list. Each line is a running process.)

4. That's too much output — filter for just your user's processes.

```
ps aux | grep $USER
```

(You'll see only processes owned by your user.)

5. Search for a specific process — let's check if SSH is running.

```
ps aux | grep sshd
```

```
root      1234  0.0  0.1  12345  6789 ?  Ss   12:00   0:00 sshd: /usr/sbin/sshd
your-user 5678  0.0  0.0   6789  1234 pts/0  S+ 12:01   0:00 grep --color=auto sshd
```

(The second line is your grep command itself — that's normal. The first line is the actual sshd process.)

6. Filter out the grep line using a common trick.

```
ps aux | grep [s]shd
```

(The bracket trick prevents grep from matching itself.)

7. Open the interactive process viewer.

```
top
```

(You'll see a live-updating list of processes. **Press q to quit.** Don't type anything else — just press q.)

8. Use htop for a better experience (we installed it in Lesson 05).

```
htop
```

(Colorful process viewer. **Press q to quit.**)

9. Check the status of the SSH service using systemctl.

```
systemctl status ssh
```

```
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset:
enabled)
     Active: active (running) since ...
```

(Press q if the output is paged. Look for "active (running)".)

10. Quick check — is SSH active? One-word answer.

```
systemctl is-active ssh
```

```
active
```

11. Check if SSH is enabled to start at boot.

```
systemctl is-enabled ssh
```

```
enabled
```

12. Check the cron service — another service that should be running.

```
systemctl status cron
```

```
● cron.service - Regular background program processing daemon
     Loaded: loaded (/lib/systemd/system/cron.service; enabled; ...)
     Active: active (running) since ...
```

13. Quick check cron.

```
systemctl is-active cron
```

```
active
```

14. List all active services.

```
systemctl list-units --type=service --state=active
```

(You'll see all services that are currently running.)

15. View recent system logs with journalctl.

```
sudo journalctl -xe | tail -n 20
```

(Shows the last 20 lines of the system journal. Output will vary.)

16. View logs for a specific service.

```
sudo journalctl -u ssh | tail -n 10
```

(Shows the last 10 SSH log entries.)

17. View logs for cron.

```
sudo journalctl -u cron | tail -n 10
```

18. Check how long the system has been running.

```
uptime
```

```
 12:00:00 up 2 days, 3:45,  1 user,  load average: 0.00, 0.01, 0.05
```

19. See how much memory is in use.

```
free -h
```

```
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       512Mi       2.1Gi       1.0Mi       1.2Gi       3.1Gi
Swap:          2.0Gi          0B       2.0Gi
```

20. Check disk space — Docker images can be large, so you'll want to know this.

```
df -h
```

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   5G   42G  11% /
...
```

(Make sure you have at least 10GB free for Docker images.)

21. Log your system status.

```
echo "System check: $(uptime)" >> ~/docker-course/notes/notes.txt
```

Checkpoint

```
systemctl is-active ssh
```

You should see `active`.

```
systemctl is-active cron
```

You should see `active`.

```
ps aux | grep [s]shd
```

You should see at least one sshd process.

```
free -h | head -n 2
```

You should see total and used memory.

```
df -h /
```

You should see at least 10GB available.

Lab

Objective: Inspect running processes with `ps`, `top`, and `htop`, and check service health with `systemctl`.

Task: Work through the drills, then run the Checkpoint commands in one terminal. Note the branch: on WSL and some VMs `ssh.service` doesn't exist — that's fine. Cron is the sanctioned substitute (see the instructor notes), so run both `is-active` checks and either one passing is a pass.

Submit: One screenshot (or pasted transcript) showing: `systemctl is-active ssh`, `systemctl is-active cron`, `free -h | head -n 2`, and `df -h /`.

Pass criteria:

- [] `systemctl is-active ssh` OR `systemctl is-active cron` prints `active` (either one passes — ssh may be inactive or missing on WSL/VMs).
- [] `free -h | head -n 2` shows the memory table.
- [] `df -h /` shows at least 10GB available.

Unlocks: 09-pipes-and-grep.md

Next up: 09-pipes-and-grep.md — Chain commands together

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Chain commands together with pipes and search files using `grep` (including -v, -i, and -r), `wc -l`, `cut`, `sort`, and `uniq`.

Lesson 09: Pipes and Grep

By the end, you will chain commands together with pipes and search through files with grep.

~15 minutes of typing

---

1. Start from home.

```
cd ~
```

2. Confirm.

```
pwd
```

```
/home/your-username
```

3. The pipe `|` sends one command's output into another command — start simple.

```
echo "hello world" | grep hello
```

```
hello world
```

(grep found "hello" in the input and printed the matching line.)

4. Search for lines that DON'T match.

```
echo "hello world" | grep -v hello
```

(No output — the only line contained "hello" and was excluded.)

5. Look at /etc/passwd — every user account is listed here.

```
cat /etc/passwd
```

(You'll see many lines. Each line is a user account.)

6. Search for your username in /etc/passwd.

```
grep $USER /etc/passwd
```

```
your-user:x:1000:1000:Your Name:/home/your-user:/bin/bash
```

7. Search for "root" in /etc/passwd.

```
grep root /etc/passwd
```

```
root:x:0:0:root:/root:/bin/bash
```

8. Count how many user accounts exist on the system.

```
wc -l /etc/passwd
```

```
32 /etc/passwd
```

(Number will vary.)

9. Same thing using a pipe.

```
cat /etc/passwd | wc -l
```

```
32
```

10. Find all accounts that use /bin/bash as their shell.

```
grep "/bin/bash" /etc/passwd
```

(You should see your user and root at minimum.)

11. Count how many use bash.

```
grep "/bin/bash" /etc/passwd | wc -l
```

```
2
```

(Number will vary.)

12. Case-insensitive search — find "ROOT" regardless of case.

```
grep -i "ROOT" /etc/passwd
```

```
root:x:0:0:root:/root:/bin/bash
```

13. Search your notes file.

```
grep "learning" ~/docker-course/notes/notes.txt
```

```
Line 3: learning cat
Line 4: learning head
Line 5: learning tail
Line 6: learning less
```

14. Count the matches.

```
grep "learning" ~/docker-course/notes/notes.txt | wc -l
```

```
4
```

15. Search recursively through a directory with grep -r.

```
grep -r "note" ~/docker-course/notes/
```

(Shows every line containing "note" in every file in the notes directory.)

16. Use ls and pipe to grep to find specific files.

```
ls /etc | grep "ssh"
```

```
ssh
```

(Shows files/directories in /etc that contain "ssh" in the name.)

17. List all installed packages and search for curl.

```
dpkg -l | grep curl
```

```
ii  curl  7.81.0-1ubuntu1  amd64  command line tool for transferring data with URL syntax
```

18. Count how many packages are installed.

```
dpkg -l | grep "^ii" | wc -l
```

```
Output will vary — you should see a number like 200-500.
```

19. Sort the lines in your notes file alphabetically.

```
sort ~/docker-course/notes/notes.txt
```

(Lines will be in alphabetical order.)

20. Get unique shells used by system accounts.

```
cat /etc/passwd | cut -d: -f7 | sort | uniq
```

```
/bin/bash
/bin/false
/bin/sync
/usr/sbin/nologin
```

(This chain: extracts the 7th field (shell), sorts them, then removes duplicates.)

21. Count how many accounts use each shell.

```
cat /etc/passwd | cut -d: -f7 | sort | uniq -c | sort -rn
```

```
     27 /usr/sbin/nologin
      2 /bin/bash
      1 /bin/sync
      1 /bin/false
```

(Numbers will vary. This is a classic pipe chain: cut → sort → uniq -c → sort.)

22. Find lines in your notes containing "check" (case-insensitive).

```
grep -i "check" ~/docker-course/notes/notes.txt
```

23. Pipe multiple commands — find your user, extract the home directory.

```
grep $USER /etc/passwd | cut -d: -f6
```

```
/home/your-username
```

24. Log your progress.

```
echo "Pipes and grep: done" >> ~/docker-course/notes/notes.txt
```

Checkpoint

```
grep $USER /etc/passwd
```

You should see your user's entry.

```
cat /etc/passwd | wc -l
```

You should see a number (total user accounts).

```
grep -r "learning" ~/docker-course/notes/
```

You should see lines containing "learning" from notes.txt.

```
dpkg -l | grep curl | wc -l
```

You should see at least `1`.

```
cat /etc/passwd | cut -d: -f7 | sort | uniq -c | sort -rn | head -n 3
```

You should see the top 3 most-used shells.

Lab

Objective: Chain commands together with pipes and search text with grep.

Task: Work through the pipe and grep drills, then run the five Checkpoint commands in one terminal.

Submit: One screenshot (or pasted transcript) of the five Checkpoint commands and their output: `grep $USER /etc/passwd`, `cat /etc/passwd | wc -l`, `grep -r "learning" ~/docker-course/notes/`, `dpkg -l | grep curl | wc -l`, `cat /etc/passwd | cut -d: -f7 | sort | uniq -c | sort -rn | head -n 3`.

Pass criteria: (account and package counts vary by machine — verify the shape of the output, not the numbers)

- [] `grep $USER /etc/passwd` shows your account's line.
- [] `cat /etc/passwd | wc -l` prints a number.
- [] `grep -r "learning"` shows the "learning" lines from notes.txt.
- [] `dpkg -l | grep curl | wc -l` prints at least `1`.
- [] The `cut | sort | uniq -c | sort -rn` chain prints the top 3 shells with counts.

Unlocks: 10-user-and-group-setup.md

Next up: 10-user-and-group-setup.md — Users and groups

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Inspect user and group membership with `id` and `groups`, add a user to a group with `sudo usermod -aG`, and refresh group membership with `newgrp`.

Lesson 10: User and Group Setup

By the end, you will understand users and groups and will have practiced the command needed to join the docker group.

~10 minutes of typing

---

1. Start from home.

```
cd ~
```

2. Confirm.

```
pwd
```

```
/home/your-username
```

3. See your user ID, group ID, and all groups you belong to.

```
id
```

```
uid=1000(your-user) gid=1000(your-user) groups=1000(your-user),4(adm),27(sudo)
```

(Output will vary. The important part: you should see `sudo` in your groups.)

4. See just your groups.

```
groups
```

```
your-user adm sudo
```

5. Look at your entry in /etc/passwd.

```
grep $USER /etc/passwd
```

```
your-user:x:1000:1000:Your Name:/home/your-user:/bin/bash
```

6. Look at the group file.

```
cat /etc/group | grep sudo
```

```
sudo:x:27:your-user
```

(This shows that your user is in the sudo group.)

7. Count how many groups exist on the system.

```
wc -l /etc/group
```

```
65 /etc/group
```

(Number will vary.)

8. List all groups your user belongs to — another way.

```
id -nG
```

```
your-user adm sudo
```

9. Check if a "docker" group exists yet.

```
grep docker /etc/group
```

(No output — the docker group doesn't exist yet. It will be created when Docker is installed in Module 02.)

10. Preview the command to add yourself to the docker group — this WILL fail right now, and that's expected.

```
sudo usermod -aG docker $USER
```

```
usermod: group 'docker' does not exist
```

(This is expected! The docker group will exist after Module 02. You're building the muscle memory now so you can type this confidently later.)

11. Practice the same command pattern with a group that DOES exist — the adm group.

```
sudo usermod -aG adm $USER
```

(No output means success. You were likely already in this group.)

12. Confirm.

```
groups
```

```
your-user adm sudo
```

(You should see `adm` in the list.)

13. Look at who's in the adm group.

```
grep adm /etc/group
```

```
adm:x:4:syslog,your-user
```

14. See all groups with your user in them.

```
grep $USER /etc/group
```

(Shows every group line that mentions your username.)

15. Check your effective user ID.

```
id -u
```

```
1000
```

(Your UID — typically 1000 for the first non-root user.)

16. Check your effective group ID.

```
id -g
```

```
1000
```

17. Remember: after adding yourself to a new group, you need to either log out and back in, or use newgrp.

```
newgrp adm
```

(This refreshes your group membership in the current shell. You'll use this after adding yourself to the docker group in Module 02.)

18. Confirm the active group changed.

```
id -gn
```

```
adm
```

19. Switch back to your default group.

```
newgrp $USER
```

20. Confirm.

```
id -gn
```

```
your-user
```

21. Log it.

```
echo "User and group setup: done" >> ~/docker-course/notes/notes.txt
```

Checkpoint

```
id
```

You should see your uid, gid, and groups including `sudo`.

```
groups
```

You should see your groups listed.

```
grep $USER /etc/group
```

You should see your user in multiple groups.

```
grep docker /etc/group
```

This should return nothing (docker group doesn't exist yet — it will after Module 02).

Lab

Objective: Read users and groups with `id` and `groups`, and rehearse the `usermod -aG` command you'll need for Docker.

Task: Work through the drills. One command is SUPPOSED to fail: `sudo usermod -aG docker $USER` errors because the docker group doesn't exist until Docker is installed in Module 02. The error is the expected result — capture it, don't fix it.

Submit: One screenshot (or pasted transcript) showing three things: the output of `id`, the `sudo usermod -aG docker $USER` command with its error `usermod: group 'docker' does not exist`, and `grep docker /etc/group` returning nothing.

Pass criteria:

- [ ] `id` shows your uid, gid, and a groups list that includes `sudo`.
- [ ] The submission includes the exact error `usermod: group 'docker' does not exist` — this failure is proof of the expected state, not a mistake.
- [ ] `grep docker /etc/group` returns no output (the docker group must NOT exist yet).

Unlocks: 11-final-checkpoint.md

Next up: 11-final-checkpoint.md — Prove you're ready for Docker

> **Lab — do it for real.** Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Live lab — Operation Uplink (“Signal Lost”)

Wednesday live lab · 6:00–8:00 PM · opens the night; Module 01 phase-gate (01:11) runs in the back half · reviews labs 01:07–10

> **Learning objective:** Re-run networking (`ip a` / `ss -tuln` / `curl` / `dig`), processes, pipes, and users/groups under a story, then sit the Module 01 phase-gate from memory.

The mission: saboteurs hit Relay Station 4 overnight and cut its uplink. Four subsystems — Antenna Array, Relay Rack, Log Archive, Crew Deck — each hold **one fragment** of the console's restart sequence. Recover all four, arm the console, and fire three test packets at the beacon. **Every station is solvable with commands from labs 01:07–10.** Some clues sit in files; some are *running*, hiding in the machine itself.

Do this before 6:00 PM — required

1. Download `setup.sh` from this topic.
2. Run `bash setup.sh` as yourself, **never** with sudo (this game never asks for a password).
3. Verify: `ls ~/uplink-hunt` shows **four station folders**.
4. Pre-flight from lab 01:07: `which dig || sudo apt install -y dnsutils`.
5. **Don't go in yet.**

```
bash setup.sh
ls ~/uplink-hunt
which dig || sudo apt install -y dnsutils
```

How the hunt works

Piece	What it means
Crews	Pairs. Each crew gets a Zoom breakout. A station is cleared only when **both** hold the fragment.
Roles (swap every station)	**Driver** shares screen and types while narrating. **Navigator** reads the briefings and calls the next command. You each drive twice.
Rule #1	Both must fire. Everything on your own machine. The final banner must come from **both terminals**.
Hints	Three hint tokens. Unlimited free hint: read the briefing again — or read what your last command printed. Machinery quiet? `bash power.sh`.

Tonight's clock

Clock	Block	What happens
6:00	Check-in	✓ in chat when `ls ~/uplink-hunt` shows four stations. Setup rescue. Crews assigned.
6:05	The hunt	60:00 on the clock. First in-story act: `bash power.sh`.
7:05	Debrief	Banners tallied. Crews teach the stations back.
7:15	Phase-gate	**Module 01 phase-gate (01:11)** — timed, no-help, from memory.
7:55	Close	Labs 01:07–11 due Sunday. Week 5: Docker arrives over the link you restored.

Train before tonight — six uplink skills

#	Can you…	Practice	From lab
1	Find your machine's own network addresses	`ip a`	01:07
2	List every port your machine is listening on	`ss -tuln`	01:07
3	Turn a name into an address — and fetch a URL	`dig +short example.com` · `curl`	01:07
4	Find one process in a crowded process table	`ps aux | grep name`	01:08
5	Chain a counting pipeline, stage by stage	`grep | cut -d: -f1 | sort | uniq -c`	01:09
6	Read your own line of the user manifest	`grep $USER /etc/passwd` · `cut -d: -f7`	01:09–10

Skill 5 is the boss fight — and the same pipeline is waiting on the phase-gate at 7:15.

Rules of the station

- Don't enter `~/uplink-hunt` before the hunt starts.
- The `.machinery` bay is sealed — snooping voids your completion code.
- Coach with words only. No web searches.
- Zero `rm -r` and zero sudo in this game.
- Wrecked something? `bash setup.sh --reset`. Machinery quiet? `bash power.sh`.

Submit. Screenshot the victory banner (completion code) and post it in Zoom chat — both crew members. Then sit lesson 01:11. Finish labs 01:07–11 in D2L by Sunday.

Learning objective: Demonstrate every skill from the module by running the cumulative command drill from memory to prove the environment is ready for Docker installation.

Lesson 11: Final Checkpoint

By the end, you will have proven that your Ubuntu environment is fully configured and that you can work in the terminal without hesitation.

~20 minutes of typing

This is a cumulative drill. Run every command below from memory. If you get stuck, go back and redo the lesson where you learned that command. Do NOT copy-paste — type every character.

System Basics

1. Print your working directory.

```
pwd
```

2. Print your username.

```
whoami
```

3. Print your hostname.

```
hostname
```

4. Show your Ubuntu version.

```
lsb_release -a
```

5. Confirm sudo works.

```
sudo whoami
```

Expected: `root`

Navigation

6. Go to the root of the filesystem and list its contents.

```
cd /
```

```
ls
```

7. Go to /etc and confirm.

```
cd /etc
```

```
pwd
```

Expected: `/etc`

8. Jump back to your previous directory.

```
cd -
```

9. Go home.

```
cd ~
```

```
pwd
```

Expected: `/home/your-username`

Files and Directories

10. List your docker-course directory with details.

```
ls -la ~/docker-course/
```

11. Show the full directory tree.

```
tree ~/docker-course/
```

12. Read your notes file.

```
cat ~/docker-course/notes/notes.txt
```

13. Show only the first 3 lines.

```
head -n 3 ~/docker-course/notes/notes.txt
```

14. Show only the last 3 lines.

```
tail -n 3 ~/docker-course/notes/notes.txt
```

15. Count the lines in your notes file.

```
wc -l ~/docker-course/notes/notes.txt
```

Permissions

16. Check the permissions on your script.

```
ls -la ~/docker-course/practice/hello.sh
```

Expected: `-rwxr-xr-x` (755)

17. Run your script.

```
~/docker-course/practice/hello.sh
```

Expected: `Hello from my script!`

Package Management

18. Update the package list.

```
sudo apt update
```

19. Verify curl is installed.

```
curl --version | head -n 1
```

20. Verify git is installed.

```
git --version
```

Networking

21. Confirm internet access.

```
ping -c 3 8.8.8.8
```

Expected: `0% packet loss`

22. Confirm DNS works.

```
ping -c 3 google.com
```

Expected: `0% packet loss`

23. Find your public IP.

```
curl ifconfig.me && echo ""
```

24. Check listening ports.

```
ss -tuln
```

Processes and Services

25. Check if SSH is running.

```
systemctl is-active ssh
```

Expected: `active`

26. Check system resource usage.

```
free -h
```

27. Check disk space.

```
df -h /
```

Expected: at least 10GB available

Pipes and Grep

28. Find your entry in /etc/passwd.

```
grep $USER /etc/passwd
```

29. Count installed packages.

```
dpkg -l | grep "^ii" | wc -l
```

30. List unique shells on the system.

```
cat /etc/passwd | cut -d: -f7 | sort | uniq
```

Users and Groups

31. Show your user ID and groups.

```
id
```

32. List your groups.

```
groups
```

33. Confirm the docker group does NOT exist yet.

```
grep docker /etc/group
```

Expected: no output (docker group will be created in Module 02)

Final Verification

Run these 5 commands. If all pass, you're ready for Module 02.

```
lsb_release -r
```

Shows `Release: 22.04` or `Release: 24.04`

```
ping -c 1 8.8.8.8
```

Shows `0% packet loss`

```
curl --version | head -n 1
```

Shows a curl version

```
git --version
```

Shows a git version

```
tree ~/docker-course/
```

Shows your course directory tree with notes, practice, and projects

---

Lab

This is the Module 01 phase-gate lab. Passing it unlocks Module 02.

Objective: Prove your Ubuntu environment is fully configured and that you can work in the terminal from memory.

Task: Run all 33 cumulative drills from memory (no copy-paste), then run the 5 Final Verification commands above in one terminal. On step 25, the Lesson 08 tolerance applies: if `ssh.service` doesn't exist on your WSL/VM setup, `systemctl is-active cron` returning `active` counts.

Submit: One screenshot or pasted transcript of the five Final Verification commands and their output: `lsb_release -r`, `ping -c 1 8.8.8.8`, `curl --version | head -n 1`, `git --version`, `tree ~/docker-course/`.

Pass criteria:

- [] `lsb_release -r` shows `Release: 22.04` or `Release: 24.04`.
- [] `ping -c 1 8.8.8.8` shows `0% packet loss`.
- [] `curl --version | head -n 1` prints a curl version line.
- [] `git --version` prints a git version line.
- [] `tree ~/docker-course/` shows the directory tree with `notes`, `practice`, and `projects`.

Unlocks: Module 02 — Docker Fundamentals (phase gate)

You're done with Module 01!

You've built the muscle memory for:

- Navigating the filesystem
- Creating and managing files
- Reading and editing files
- Updating your system
- Installing software
- Setting permissions
- Diagnosing networking
- Inspecting processes and services
- Using pipes and grep
- Managing users and groups

Next: Module 02 — Docker Installation. Everything you typed in this module was preparation for what comes next.

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.